

Mini Project 2 Big Data Report

Ziad Moutaz 202201252, Mohamed Hozien 202201507

April 2026

1 Problem Statement

The aviation industry generates enormous volumes of operational data daily, yet flight delays remain one of the most persistent and costly inefficiencies in air travel. In 2019–2023, tens of millions of US domestic flights were affected by delays attributable to a range of causes including carrier operations, weather events, National Airspace System congestion, security incidents, and late-arriving aircraft.

Research Question: *What drives flight delay severity across US airlines and routes, and how do delay causes propagate over time?*

Specifically, we investigate three analytical dimensions:

- **Trend Analysis** – Temporal delay patterns using 7-day moving averages and cumulative cancellation curves to identify seasonal spikes and long-term airline degradation trends.
- **Anomaly Detection** – Statistical identification of route-level delay outliers (departures exceeding $\mu + 3\sigma$ of the route mean), enabling targeted infrastructure or scheduling interventions.
- **Root Cause Attribution** – Decomposing aggregate delay minutes by cause (carrier, weather, NAS, security, late aircraft) per airline to prioritise improvement efforts.

The dataset’s 3 million records and 32 mixed-type columns provide the scale and richness needed to stress-test Apache Spark’s distributed execution model, compare its three programming APIs (SQL, DataFrame, RDD), and demonstrate the concrete performance gains delivered by the Catalyst Optimizer and Tungsten execution engine.

2 Dataset Description

We used US Flight Delay and Cancellation Data from Kaggle, it contains 3M records, and it’s schema is defined as in the following table:

Column Name	Data Type
FL_DATE	Date
AIRLINE	String
AIRLINE_DOT	String
AIRLINE_CODE	String
DOT_CODE	Integer
FL_NUMBER	Integer
ORIGIN	String
ORIGIN_CITY	String
DEST	String
DEST_CITY	String
CRS_DEP_TIME	Integer
DEP_TIME	Double
DEP_DELAY	Double
TAXI_OUT	Double
WHEELS_OFF	Double
WHEELS_ON	Double
TAXI_IN	Double
CRS_ARR_TIME	Integer
ARR_TIME	Double
ARR_DELAY	Double
CANCELLED	Double
CANCELLATION_CODE	String
DIVERTED	Double
CRS_ELAPSED_TIME	Double
ELAPSED_TIME	Double
AIR_TIME	Double
DISTANCE	Double
DELAY_DUE_CARRIER	Double
DELAY_DUE_WEATHER	Double
DELAY_DUE_NAS	Double
DELAY_DUE_SECURITY	Double
DELAY_DUE_LATE_AIRCRAFT	Double

Table 1: Schema of Flights Dataset

2.1 Justification

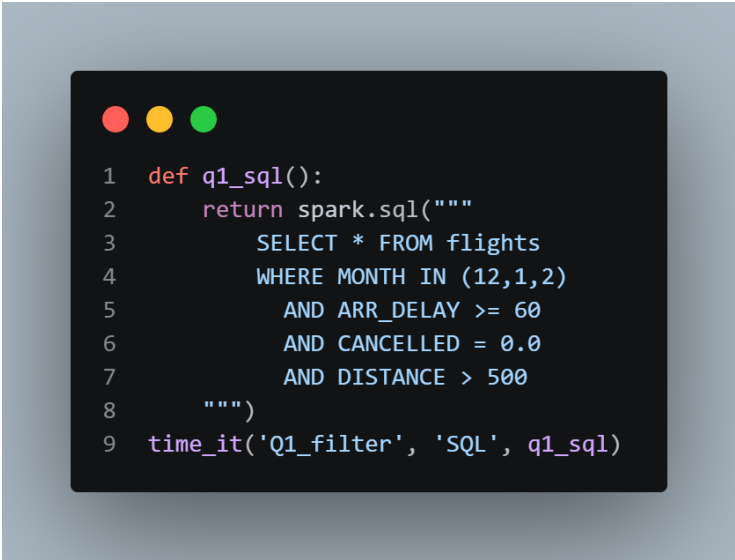
This data is appropriate to be distributed processed due to the presence of 3 million records, which is efficient to be processed with the help of such frameworks as Spark to carry out complex operations (joins, aggregations, window functions, etc.). Moreover, it contains 32 mixed-columns with both numerical, categorical and date values, which can be helpful in testing Spark SQL and query optimization abilities. It also inherently supports join operations to dimension tables, like airlines and airports, and time-related capabilities allow more complex analytics, like window functions, moving averages, and cumula-

tive computations.

3 Queries Implemented

3.1 Q1

3.1.1 SQL



```
1 def q1_sql():
2     return spark.sql("""
3         SELECT * FROM flights
4         WHERE MONTH IN (12,1,2)
5             AND ARR_DELAY >= 60
6             AND CANCELLED = 0.0
7             AND DISTANCE > 500
8     """)
9 time_it('Q1_filter', 'SQL', q1_sql)
```

Figure 1: Q1 - SQL Implementation

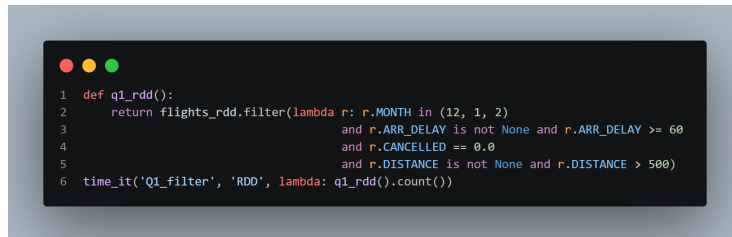
3.1.2 Dataframe



```
1 def q1_df():
2     return (flights
3             .filter(F.col('MONTH').isin(12, 1, 2))
4             .filter(F.col('ARR_DELAY') >= 60)
5             .filter(F.col('CANCELLED') == 0.0)
6             .filter(F.col('DISTANCE') > 500))
7 q1_df_res = time_it('Q1_filter', 'DataFrame', q1_df)
8
```

Figure 2: Q1 - DataFrame Implementation

3.1.3 RDD



```
1 def q1_rdd():
2     return flights_rdd.filter(lambda r: r.MONTH in (12, 1, 2)
3                                and r.ARR_DELAY is not None and r.ARR_DELAY >= 60
4                                and r.CANCELLED == 0.0
5                                and r.DISTANCE is not None and r.DISTANCE > 500)
6 time_it('Q1_filter', 'RDD', lambda: q1_rdd().count())
```

Figure 3: Q1 - RDD Implementation

3.1.4 Explain

The **parsed logical plan** expresses Q1 as a chain of four **Filter** nodes (long-haul, severe arrival delay, winter month, non-cancelled) above a **CSV Relation**. The **optimized logical plan** collapses the four predicates into a single conjunction and adds **isnotnull** guards. The **physical plan** is a single-stage **FileScan csv** with a **PushedFilters** clause carrying every predicate, followed by an in-memory **Project** of three columns – no **Exchange**, no shuffle.

```

-- Parsed logical plan ==
Filter <-> (DISTANCE > 500)
+- Filter (CANCELLED#20 = 0.0)
  +- Filter (ARR_DELAY#19 >= cast(60 as double))
    +- Filter MONTH#4 IN (12,1,2)
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
        +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
          +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
            +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11]

-- Analyzed logical plan ==
FL_DATE: date, AIRLINE: string, AIRLINE_DOT: string, AIRLINE_CODE: string, DOT_CODE: int, FL_NUMBER: int, ORIGIN: string, ORIGIN_CITY: string, DEST: string, DEST_CITY: string
Filter (DISTANCE#26 > cast(500 as double))
+- Filter (CANCELLED#20 = 0.0)
  +- Filter (ARR_DELAY#19 >= cast(60 as double))
    +- Filter MONTH#4 IN (12,1,2)
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
        +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
          +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
            +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11]

-- Optimized logical plan ==
Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+- Filter (((isnotnull(ARR_DELAY#19) and isnotnull(CANCELLED#20)) and isnotnull(DISTANCE#26)) and ((month(FL_DATE#0) in (12,1,2) and (ARR_DELAY#19 >= 60.0)) and ((CANCELLED#20 = 0.0) or (ARR_DELAY#19 >= 60.0))))
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12,ARR_DELAY#19]

-- physical plan ==
*(1) project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+- *(1) filter ((((((isnotnull(ARR_DELAY#19) and isnotnull(CANCELLED#20)) and isnotnull(DISTANCE#26)) and month(FL_DATE#0) in (12,1,2)) and (ARR_DELAY#19 >= 60.0)) and (CANCELLED#20 = 0.0) or (ARR_DELAY#19 >= 60.0))))
+- FileScan csv [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12,ARR_DELAY#19]

```

Figure 4: Q1 Explain

3.2 Q2

3.2.1 SQL

```

1  def q2_sql():
2      return spark.sql("""
3          SELECT AIRLINE_CODE,
4                  COUNT(*)           AS total_flights,
5                  SUM(CANCELLED)     AS cancellations,
6                  AVG(ARR_DELAY)     AS avg_arr_delay,
7                  MAX(ARR_DELAY)     AS max_arr_delay,
8                  MIN(ARR_DELAY)     AS min_arr_delay
9          FROM flights
10         GROUP BY AIRLINE_CODE
11         """)
12  time_it('Q2_aggregate', 'SQL', q2_sql)
13

```

Figure 5: Q2 - SQL Implementation

3.2.2 Dataframe

```
1 def q2_df():
2     return (flights.groupBy('AIRLINE_CODE')
3             .agg(F.count('*').alias('total_flights'),
4                 F.sum('CANCELLED').alias('cancellations'),
5                 F.avg('ARR_DELAY').alias('avg_arr_delay'),
6                 F.max('ARR_DELAY').alias('max_arr_delay'),
7                 F.min('ARR_DELAY').alias('min_arr_delay')))
8 q2_df_res = time_it('Q2_aggregate', 'DataFrame', q2_df)
9
```

Figure 6: Q2 - DataFrame Implementation

3.2.3 RDD

```
1 def q2_rdd():
2     kv = flights_rdd.map(lambda r: (r.AIRLINE_CODE, (
3         1,
4         float(r.CANCELLED or 0),
5         float(r.ARR_DELAY) if r.ARR_DELAY is not None else 0.0,
6         1 if r.ARR_DELAY is not None else 0,
7         float(r.ARR_DELAY) if r.ARR_DELAY is not None else float('-inf'),
8         float(r.ARR_DELAY) if r.ARR_DELAY is not None else float('inf'))))
9     reduced = kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1], a[2]+b[2], a[3]+b[3], max(a[4], b[4]), min(a[5], b[5])))
10    return reduced.map(lambda x: (x[0], x[1][0], x[1][1],
11                                   x[1][2]/x[1][3] if x[1][3] else None, x[1][4], x[1][5]))
12 time_it('Q2_aggregate', 'RDD', lambda: q2_rdd().count())
13
```

Figure 7: Q2 - RDD Implementation

3.2.4 Explain

The **parsed logical plan** groups the full row-set by AIRLINE_CODE and computes avg(ARR_DELAY). The **optimized logical plan** prunes the schema down to the two needed columns and pushes an isnotnull(ARR_DELAY) filter into the file scan. The **physical plan** adopts the canonical two-stage HashAggregate: a partial aggregation per partition, an Exchange hashpartitioning(AIRLINE_CODE, 200) shuffle, then a final HashAggregate to merge the partials.

```

+ Parsed logical Plan ==
Aggregate [AIRLINE_CODE], [AIRLINE_CODE, 'count(*) AS total_flights#258', 'sum('CANCELLED') AS cancellations#259, 'avg('ARR_DELAY') AS avg_arr_delay#260, 'max('ARR_DELAY') AS max_arr_delay#261, 'min('ARR_DELAY') AS min_arr_delay#262']
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

+ Analyzed logical Plan ==
AIRLINE_CODE: string, total_flights: bigint, cancellations: double, avg_arr_delay: double, max_arr_delay: double, min_arr_delay: double
Aggregate [AIRLINE_CODE#3], [AIRLINE_CODE#3, count(1) AS total_flights#258, sum(CANCELLED#20) AS cancellations#259, avg(ARR_DELAY#19) AS avg_arr_delay#260, max(ARR_DELAY#19) AS max_arr_delay#261, min(ARR_DELAY#19) AS min_arr_delay#262]
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

+ Optimized logical Plan ==
Aggregate [AIRLINE_CODE#3], [AIRLINE_CODE#3, count(1) AS total_flights#258, sum(CANCELLED#20) AS cancellations#259, avg(ARR_DELAY#19) AS avg_arr_delay#260, max(ARR_DELAY#19) AS max_arr_delay#261, min(ARR_DELAY#19) AS min_arr_delay#262]
+ Project [AIRLINE_CODE#3, ARR_DELAY#19, CANCELLED#20]
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

+ Physical Plan ==
daptivesparkplan isFinalPlan=false
HashAggregate(keys=[AIRLINE_CODE#3], functions=[count(1), sum(CANCELLED#20), avg(ARR_DELAY#19), max(ARR_DELAY#19), min(ARR_DELAY#19)], output=[AIRLINE_CODE#3, total_flights#258, cancellations#259, avg_arr_delay#260, max_arr_delay#261, min_arr_delay#262])
+ Exchange hashpartitioning(AIRLINE_CODE#3, 200), ENSURE_REQUIREMENTS, [plan_id=587]
+ HashAggregate(keys=[AIRLINE_CODE#3], functions=[partial count(1), partial sum(CANCELLED#20), partial avg(ARR_DELAY#19), partial max(ARR_DELAY#19), partial min(ARR_DELAY#19)], output=[AIRLINE_CODE#3, partial_total_flights#258, partial_cancellations#259, partial_avg_arr_delay#260, partial_max_arr_delay#261, partial_min_arr_delay#262])
+ FileScan csv [AIRLINE_CODE#3,ARR_DELAY#19,CANCELLED#20] Batched: false, DataFilters: [], format: CSV, location: InMemoryFileIndex(1 path)[file:/content/flight/]

```

Figure 8: Q2 Explain

3.3 Q3

3.3.1 SQL

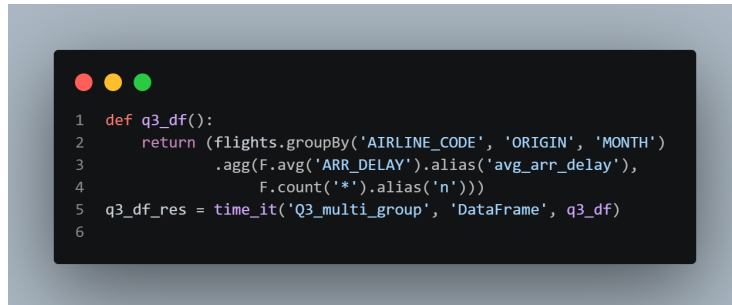
```

1 def q3_sql():
2     return spark.sql("""
3         SELECT AIRLINE_CODE, ORIGIN, MONTH,
4               AVG(ARR_DELAY) AS avg_arr_delay,
5               COUNT(*)      AS n
6         FROM flights
7         WHERE ARR_DELAY IS NOT NULL
8         GROUP BY AIRLINE_CODE, ORIGIN, MONTH
9     """)
10 time_it('Q3_multi_group', 'SQL', q3_sql)

```

Figure 9: Q3 - SQL Implementation

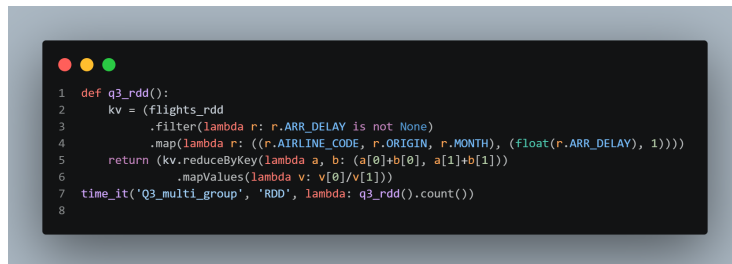
3.3.2 Dataframe



```
1 def q3_df():
2     return (flights.groupBy('AIRLINE_CODE', 'ORIGIN', 'MONTH')
3             .agg(F.avg('ARR_DELAY').alias('avg_arr_delay'),
4                 F.count('*').alias('n')))
5 q3_df_res = time_it('Q3_multi_group', 'DataFrame', q3_df)
6
```

Figure 10: Q3 - DataFrame Implementation

3.3.3 RDD



```
1 def q3_rdd():
2     kv = (flights_rdd
3           .filter(lambda r: r.ARR_DELAY is not None)
4           .map(lambda r: ((r.AIRLINE_CODE, r.ORIGIN, r.MONTH), (float(r.ARR_DELAY), 1))))
5     return (kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
6            .mapValues(lambda v: v[0]/v[1]))
7 time_it('Q3_multi_group', 'RDD', lambda: q3_rdd().count())
8
```

Figure 11: Q3 - RDD Implementation

3.3.4 Q3 Explain

The **parsed logical plan** derives a MONTH column then groups by three keys (AIRLINE_CODE, ORIGIN, MONTH). The **optimized logical plan** pushes the MONTH computation below the aggregation, prunes columns to the four needed, and inserts `isnotnull` filters at scan. The **physical plan** exhibits two-stage `HashAggregate` with a wide `Exchange hashpartitioning(AIRLINE_CODE, ORIGIN, MONTH, 200)` – the high-cardinality composite key makes shuffle the dominant cost.


```

== Parsed Logical Plan ==
Aggregate [AIRLINE_CODE, 'ORIGIN', 'MONTH'], [AIRLINE_CODE, 'ORIGIN', 'MONTH', 'avg(ARR_DELAY) AS avg_arr_delay#379, 'count(*) AS nb#380]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
   +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
      +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

== Analyzed Logical Plan ==
Aggregate [AIRLINE_CODE#3, ORIGIN#6, MONTH#34], [AIRLINE_CODE#3, ORIGIN#6, MONTH#34, avg(ARR_DELAY#19) AS avg_arr_delay#379, count(1) AS nb#380]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
   +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DELAY#12]
      +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

== Optimized Logical Plan ==
Aggregate [AIRLINE_CODE#3, ORIGIN#6, MONTH#34], [AIRLINE_CODE#3, ORIGIN#6, MONTH#34, avg(ARR_DELAY#19) AS avg_arr_delay#379, count(1) AS nb#380]
+- Project [AIRLINE_CODE#3, ORIGIN#6, ARR_DELAY#19, month(FL_DATE#0) AS MONTH#34]
   +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[AIRLINE_CODE#3, ORIGIN#6, MONTH#34], functions=[avg(ARR_DELAY#19), count(1)], output=[AIRLINE_CODE#3, ORIGIN#6, MONTH#34, avg_arr_delay#379, nb#380])
   +- Exchange hashpartitioning(AIRLINE_CODE#3, ORIGIN#6, MONTH#34, 200), ENSURE_REQUIREMENTS, [plan_id=795]
      +- HashAggregate(keys=[AIRLINE_CODE#3, ORIGIN#6, MONTH#34], functions=[partial_avg(ARR_DELAY#19), partial_count(1)], output=[AIRLINE_CODE#3, ORIGIN#6, MONTH#34, sum_avg_arr_delay#379, sum_nb#380])
         +- Project [AIRLINE_CODE#3, ORIGIN#6, ARR_DELAY#19, month(FL_DATE#0) AS MONTH#34]
            +- FileScan csv [FL_DATE#0,AIRLINE_CODE#3,ORIGIN#6,ARR_DELAY#19] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryfileIndex(1 paths)[file:/content

```

Figure 12: Q3 Explain

3.4 Q4

3.4.1 SQL

```

1 def q4_sql():
2     return spark.sql("""
3         SELECT ORIGIN, DEST,
4             AVG(ARR_DELAY) AS avg_delay,
5             COUNT(*)      AS flights
6         FROM flights
7         GROUP BY ORIGIN, DEST
8         HAVING COUNT(*) > 100
9         ORDER BY avg_delay DESC
10        LIMIT 20
11    """)
12 time_it('Q4_top20_routes', 'SQL', q4_sql)
13

```

Figure 13: Q4 - SQL Implementation

3.4.2 Dataframe

```
1 def q4_df():
2     return (flights.groupBy('ORIGIN', 'DEST')
3             .agg(F.avg('ARR_DELAY').alias('avg_delay'),
4                 F.count('*').alias('flights'))
5             .filter('flights > 100')
6             .orderBy(F.desc('avg_delay'))
7             .limit(20))
8 q4_df_res = time_it('Q4_top20_routes', 'DataFrame', q4_df)
9
```

Figure 14: Q4 - DataFrame Implementation

3.4.3 RDD

```
1 def q4_rdd():
2     kv = (flights_rdd
3          .filter(lambda r: r.ARR_DELAY is not None)
4          .map(lambda r: ((r.ORIGIN, r.DEST), (float(r.ARR_DELAY), 1))))
5     agg = kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])).mapValues(lambda v: v[0]/v[1])
6     return agg.takeOrdered(20, key=lambda x: -x[1])
7 time_it('Q4_top20_routes', 'RDD', lambda: len(q4_rdd()))
8
```

Figure 15: Q4 - RDD Implementation

3.4.4 Q4 Explain

The **parsed logical plan** groups by (ORIGIN, DEST), filters `flights > 100`, sorts by average delay descending, and limits to 20. The **optimized logical plan** prunes to three columns and reorders the post-aggregation Filter after the HashAggregate (it cannot be pushed below the group-by). The **physical plan** replaces the naive Sort + Limit with a single TakeOrderedAndProject(20) operator that performs a partial top-20 per partition before a final merge – avoiding a full shuffle-sort on 24k aggregated rows.

```

== Parsed logical Plan ==
GlobalLimit 20
+- LocalLimit 20
  +- Sort [avg_delay#453 DESC NULLS LAST], true
  +- Filter (flights#454L > cast(100 as bigint))
  +- Aggregate [ORIGIN#6, DEST#8], [ORIGIN#6, DEST#8, avg(ARR_DELAY#19) AS avg_delay#453, count(1) AS flights#454L]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10]

== Analyzed logical Plan ==
ORIGIN: string, DEST: string, avg delay: double, flights: bigint
GlobalLimit 20
+- LocalLimit 20
  +- Sort [avg_delay#453 DESC NULLS LAST], true
  +- Filter (flights#454L > cast(100 as bigint))
  +- Aggregate [ORIGIN#6, DEST#8], [ORIGIN#6, DEST#8, avg(ARR_DELAY#19) AS avg_delay#453, count(1) AS flights#454L]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10]
    +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10]

== Optimized logical Plan ==
GlobalLimit 20
+- LocalLimit 20
  +- Sort [avg_delay#453 DESC NULLS LAST], true
  +- Filter (flights#454L > 100)
  +- Aggregate [ORIGIN#6, DEST#8], [ORIGIN#6, DEST#8, avg(ARR_DELAY#19) AS avg_delay#453, count(1) AS flights#454L]
    +- Project [ORIGIN#6, DEST#8, ARR_DELAY#19]
    +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10]

```

Figure 16: Q4 Explain

3.5 Q5

3.5.1 SQL

```

1 def q5_sql():
2     return spark.sql("""
3         WITH daily AS (
4             SELECT AIRLINE_CODE, FL_DATE, AVG(ARR_DELAY) AS daily_avg
5             FROM flights WHERE ARR_DELAY IS NOT NULL
6             GROUP BY AIRLINE_CODE, FL_DATE
7         )
8         SELECT AIRLINE_CODE, FL_DATE, daily_avg,
9             AVG(daily_avg) OVER (PARTITION BY AIRLINE_CODE
10                                ORDER BY FL_DATE
11                                ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
12         FROM daily
13     """)
14 time_it('Q5_moving_avg', 'SQL', q5_sql)
15

```

Figure 17: Q5 - SQL Implementation

3.5.2 Dataframe

```
1 def q5_df():
2     daily = (flights.filter(F.col('ARR_DELAY').isNotNull())
3              .groupBy('AIRLINE_CODE', 'FL_DATE')
4              .agg(F.avg('ARR_DELAY').alias('daily_avg')))
5     w = Window.partitionBy('AIRLINE_CODE').orderBy('FL_DATE').rowsBetween(-6, 0)
6     return daily.withColumn('ma7', F.avg('daily_avg').over(w))
7 q5_df_res = time_it('Q5_moving_avg', 'DataFrame', q5_df)
8
```

Figure 18: Q5 - DataFrame Implementation

3.5.3 RDD

```
1 def q5_rdd():
2     daily = flights_rdd
3         .filter(lambda r: r.ARR_DELAY is not None)
4         .map(lambda r: ((r.AIRLINE_CODE, r.FL_DATE), (float(r.ARR_DELAY), 1)))
5         .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
6         .map(lambda x: (x[0][0], (x[0][1], x[1][0]/x[1][1])))
7     grouped = daily.groupByKey().mapValues(lambda it: sorted(it))
8     def rolling(pairs):
9         out = []
10        for i in range(len(pairs)):
11            window = pairs[max(0, i-6):i+1]
12            out.append((pairs[i][0], sum(p[1] for p in window)/len(window)))
13        return out
14    return grouped.flatMapValues(rolling)
15 time_it('Q5_moving_avg', 'RDD', lambda: q5_rdd().count())
16
```

Figure 19: Q5 - RDD Implementation

3.5.4 Q5 Explain

The **parsed logical plan** contains a `groupBy(AIRLINE_CODE, FL_DATE)` averaging `ARR_DELAY` feeding a `Window` with `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`. The **optimized logical plan** prunes to three columns and pushes the `isNotNull(ARR_DELAY)` guard to the file scan. The **physical plan** chains two `Exchange` stages: first `hashpartitioning(AIRLINE_CODE, FL_DATE, 200)` for the aggregation, then a re-partition to `hashpartitioning(AIRLINE_CODE, 200)` followed by a `Sort(FL_DATE)` so the `Window` operator can sweep each partition in order.

```

== Parsed logical Plan ==
Project [unresolvedstagecolumns(ma7, 'avg('daily_avg' window(specdefinition('AIRLINE_CODE', 'FL_DATE ASC NULLS FIRST', specifiedwindowframe(rowframe, -6, currentrowid()),
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, avg(ARR_DELAY#19) AS daily_avg#559]
+- Filter [isnotnull(ARR_DELAY#19)]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,0]

== Analyzed logical Plan ==
AIRLINE_CODE: string, FL_DATE: date, daily_avg: double, ma7: double
Project [AIRLINE_CODE#3, FL_DATE#0, daily_avg#559, ma7#596]
+- Project [AIRLINE_CODE#3, FL_DATE#0, daily_avg#559, ma7#596]
+- Window [avg(daily_avg#559) window(specdefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, -6, currentrowid())) AS ma7#596), [AIRLINE_CODE#3,
+- Project [AIRLINE_CODE#3, FL_DATE#0, daily_avg#559]
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, avg(ARR_DELAY#19) AS daily_avg#559]
+- Filter [isnotnull(ARR_DELAY#19)]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11,
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,0]

== Optimized logical Plan ==
Window [avg(daily_avg#559) window(specdefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, -6, currentrowid())) AS ma7#596), [AIRLINE_CODE#3]
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, avg(ARR_DELAY#19) AS daily_avg#559]
+- Project [FL_DATE#0, AIRLINE_CODE#3, ARR_DELAY#19]
+- Filter [isnotnull(ARR_DELAY#19)]
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,0]

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Window [avg(daily_avg#559) window(specdefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, -6, currentrowid())) AS ma7#596), [AIRLINE_CODE#3]]

```

Figure 20: Q5 Explain

3.6 Q6

3.6.1 SQL

```

1 def q6_sql():
2     return spark.sql("""
3         WITH daily AS (
4             SELECT AIRLINE_CODE, FL_DATE, SUM(CANCELLED) AS daily_cancel
5             FROM flights GROUP BY AIRLINE_CODE, FL_DATE
6         )
7         SELECT AIRLINE_CODE, FL_DATE, daily_cancel,
8             SUM(daily_cancel) OVER (PARTITION BY AIRLINE_CODE
9                                     ORDER BY FL_DATE
10                                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_cancel
11         FROM daily
12     """)
13 time_it('Q6_cumulative', 'SQL', q6_sql)
14

```

Figure 21: Q6 - SQL Implementation

3.6.2 Dataframe

```
1 def q6_df():
2     daily = (flights.groupBy('AIRLINE_CODE', 'FL_DATE')
3               .agg(F.sum('CANCELLED').alias('daily_cancel')))
4     w = (Window.partitionBy('AIRLINE_CODE').orderBy('FL_DATE')
5           .rowsBetween(Window.unboundedPreceding, Window.currentRow))
6     return daily.withColumn('cum_cancel', F.sum('daily_cancel').over(w))
7 q6_df_res = time_it('Q6_cumulative', 'DataFrame', q6_df)
8
```

Figure 22: Q6 - DataFrame Implementation

3.6.3 RDD

```
1 def q6_rdd():
2     daily = (flights_rdd
3               .map(lambda r: ((r.AIRLINE_CODE, r.FL_DATE), float(r.CANCELLED or 0)))
4               .reduceByKey(lambda a, b: a + b)
5               .map(lambda x: (x[0][0], (x[0][1], x[1]))))
6     grouped = daily.groupByKey().mapValues(lambda it: sorted(it))
7     def cumul(pairs):
8         total = 0.0; out = []
9         for d, c in pairs:
10             total += c; out.append((d, total))
11     return out
12     return grouped.flatMapValues(cumul)
13 time_it('Q6_cumulative', 'RDD', lambda: q6_rdd().count())
14
```

Figure 23: Q6 - RDD Implementation

3.6.4 Q6 Explain

The **parsed logical plan** sums CANCELLED per (AIRLINE_CODE, FL_DATE) then applies an unbounded preceding window for the cumulative count. The **optimized logical plan** prunes to those three columns and folds away unused projections. The **physical plan** mirrors Q5 – two Exchange stages plus a Sort(FL_DATE) – but the window operator uses a *growing* frame, so memory cost per partition increases linearly with rows; Catalyst still uses a single streaming pass rather than recomputing the prefix sum from scratch.

```

== Parsed Logical Plan ==
Project [unresolvedtablewithcolumns(cum_cancel, 'sum('daily_cancel' windowSpecDefinition('AIRLINE_CODE', 'FL_DATE ASC NULLS FIRST, specifiedwindowframe(rowframe, unboundedpre
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, sum(CANCELLED#20) AS daily_cancel#654]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, D
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, D
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DE

== Analyzed logical Plan ==
AIRLINE_CODE: string, FL_DATE: date, daily_cancel: double, cum_cancel: double
Project [AIRLINE_CODE#3, FL_DATE#0, daily_cancel#654, cum_cancel#691]
+- Project [AIRLINE_CODE#3, FL_DATE#0, daily_cancel#654, cum_cancel#691]
+- Window [sum(daily_cancel#654) windowSpecDefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, unboundedprecedings$(), currentrow$())) AS
+- Project [AIRLINE_CODE#3, FL_DATE#0, daily_cancel#654]
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, sum(CANCELLED#20) AS daily_cancel#654]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_T
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DE
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

== Optimized Logical Plan ==
Window [sum(daily_cancel#654) windowSpecDefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, unboundedprecedings$(), currentrow$())) AS cum_cancel#691]
+- Aggregate [AIRLINE_CODE#3, FL_DATE#0], [AIRLINE_CODE#3, FL_DATE#0, sum(CANCELLED#20) AS daily_cancel#654]
+- Project [FL_DATE#0, AIRLINE_CODE#3, CANCELLED#20]
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_DELAY#12]

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Window [sum(daily_cancel#654) windowSpecDefinition(AIRLINE_CODE#3, FL_DATE#0 ASC NULLS FIRST, specifiedwindowframe(rowframe, unboundedprecedings$(), currentrow$())) AS cum_cancel#691]
+- Sort [AIRLINE_CODE#3 ASC NULLS FIRST, FL_DATE#0 ASC NULLS FIRST], False, 0
+- Exchange hashpartitioning(AIRLINE_CODE#3, 200), ENSURE_REQUIREMENTS, [plan_id=1612]
+- HashAggregate(keys=[AIRLINE_CODE#3, FL_DATE#0], functions=[sum(CANCELLED#20)], output=[AIRLINE_CODE#3, FL_DATE#0, daily_cancel#654])

```

Figure 24: Q6 Explain

3.7 Q7

3.7.1 SQL

```

1 def q7_sql():
2     return spark.sql("""
3         WITH monthly AS (
4             SELECT YEAR, MONTH, AIRLINE_CODE,
5                 SUM(CASE WHEN ARR_DELAY <= 15 THEN 1 ELSE 0 END) / COUNT(*) AS on_time_rate
6             FROM Flights WHERE ARR_DELAY IS NOT NULL
7             GROUP BY YEAR, MONTH, AIRLINE_CODE
8         )
9         SELECT YEAR, MONTH, AIRLINE_CODE, on_time_rate,
10             RANK() OVER (PARTITION BY YEAR, MONTH ORDER BY on_time_rate DESC) AS rnk
11         FROM monthly
12     """)
13     time_it('Q7_rank', 'SQL', q7_sql)
14

```

Figure 25: Q7 - SQL Implementation

3.7.2 Dataframe

```

1 def q7_df():
2     monthly = (flights.filter(F.col('ARR_DELAY').isNotNull())
3                 .groupBy('YEAR', 'MONTH', 'AIRLINE_CODE')
4                 .agg((F.sum((F.col('ARR_DELAY') <= 15).cast('int')) / F.count('*')).alias('on_time_rate')))
5     w = Window.partitionBy('YEAR', 'MONTH').orderBy(F.desc('on_time_rate'))
6     return monthly.withColumn('rnk', F.rank().over(w))
7     q7_df_res = time_it('Q7_rank', 'DataFrame', q7_df)

```

Figure 26: Q7 - DataFrame Implementation

3.7.3 RDD

```

1  def q7_rdd():
2    kv = (flights_rdd
3          .filter(lambda r: r.ARR_DELAY is not None)
4          .map(lambda r: ((r.YEAR, r.MONTH, r.AIRLINE_CODE),
5                          (1, 1 if r.ARR_DELAY <= 15 else 0))))
6    agg = (kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
7          .map(lambda x: (x[0][0], x[0][1], x[0][2], x[1][1]/x[1][0])))
8    grouped = agg.map(lambda x: ((x[0], x[1]), (x[2], x[3]))).groupByKey()
9    def rank(rows):
10     s = sorted(rows, key=lambda r: -r[1])
11     return [(airline, rate, i+1) for i, (airline, rate) in enumerate(s)]
12    return grouped.flatMapValues(rank)
13    time_it('Q7_rank', 'RDD', lambda: q7_rdd().count())

```

Figure 27: Q7 - RDD Implementation

3.7.4 Q7 Explain

The **parsed logical plan** derives YEAR/MONTH, computes the on-time rate as a ratio of conditional sums per (YEAR, MONTH, AIRLINE_CODE), then applies a RANK() window partitioned by (YEAR, MONTH). The **optimized logical plan** prunes to two source columns plus the two derived ones and pushes isNotNull(ARR_DELAY). The **physical plan** runs a two-stage HashAggregate via hashpartitioning (YEAR, MONTH, AIRLINE_CODE, 200), then re-shuffles to hashpartitioning (YEAR, MONTH, 200) and Sorts by descending on-time-rate before the streaming RANK window.

```

== Parsed logical plan ==
Project [unresolvedstarwithcolumns(rnk, 'rank()' windowspecdefinition('YEAR', 'MONTH', 'on time rate DESC NULLS LAST, unspecifiedframe$', None))
+- Aggregate [YEAR#33, MONTH#34, AIRLINE_CODE#3], [YEAR#33, MONTH#34, AIRLINE_CODE#3], (cast(sum(cast((ARR_DELAY#19 <= cast(15 as double)) as int)) as double) / cast(count
+- Filter isNotNull(ARR_DELAY#19)
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11]

== Analyzed logical plan ==
YEAR: int, MONTH: int, AIRLINE_CODE: string, on time rate: double, rnk: int
Project [YEAR#33, MONTH#34, AIRLINE_CODE#3, on time rate#719, rnk#757]
+- Project [YEAR#33, MONTH#34, AIRLINE_CODE#3, on time rate#719, rnk#757, rnk#757]
+- Window [rank(on time rate#719) windowspecdefinition(YEAR#33, MONTH#34, on time rate#719 DESC NULLS LAST, specifiedwindowframe(rowframe, unboundedpreceding$(), currentrow))
+- Project [YEAR#33, MONTH#34, AIRLINE_CODE#3, on time rate#719]
+- Aggregate [YEAR#33, MONTH#34, AIRLINE_CODE#3], [YEAR#33, MONTH#34, AIRLINE_CODE#3], (cast(sum(cast((ARR_DELAY#19 <= cast(15 as double)) as int)) as double) / cast(count(1) as double)
+- Filter isNotNull(ARR_DELAY#19)
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11]
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11]

== Optimized logical plan ==
Window [rank(on time rate#719) windowspecdefinition(YEAR#33, MONTH#34, on time rate#719 DESC NULLS LAST, specifiedwindowframe(rowframe, unboundedpreceding$(), currentrow))
+- Aggregate [YEAR#33, MONTH#34, AIRLINE_CODE#3], [YEAR#33, MONTH#34, AIRLINE_CODE#3], (cast(sum(cast((ARR_DELAY#19 <= 15.0) as int)) as double) / cast(count(1) as double)
+- Project [AIRLINE_CODE#3, ARR_DELAY#19, year(FL_DATE#0) AS YEAR#33, month(FL_DATE#0) AS MONTH#34]
+- Filter isNotNull(ARR_DELAY#19)
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_TIME#11]

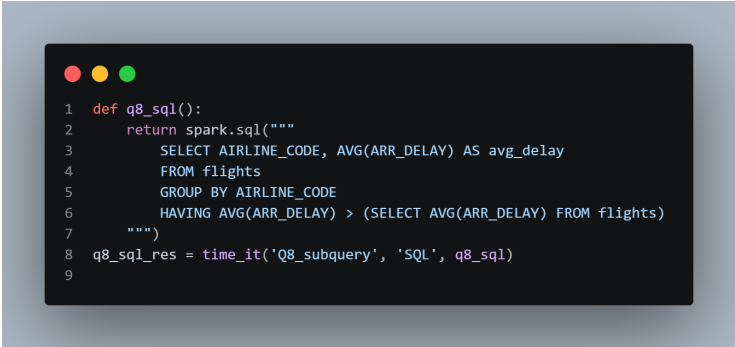
== Physical plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Window [rank(on time rate#719) windowspecdefinition(YEAR#33, MONTH#34, on time rate#719 DESC NULLS LAST, specifiedwindowframe(rowframe, unboundedpreceding$(), currentrow))

```

Figure 28: Q7 Explain

3.8 Q8

3.8.1 SQL



```
1 def q8_sql():
2     return spark.sql("""
3         SELECT AIRLINE_CODE, AVG(ARR_DELAY) AS avg_delay
4         FROM flights
5         GROUP BY AIRLINE_CODE
6         HAVING AVG(ARR_DELAY) > (SELECT AVG(ARR_DELAY) FROM flights)
7     """)
8 q8_sql_res = time_it('Q8_subquery', 'SQL', q8_sql)
9
```

Figure 29: Q8 - SQL Implementation

3.8.2 Dataframe



```
1 def q8_df():
2     overall = flights.agg(F.avg('ARR_DELAY').alias('o')).first()['o']
3     return (flights.groupBy('AIRLINE_CODE')
4             .agg(F.avg('ARR_DELAY').alias('avg_delay'))
5             .filter(F.col('avg_delay') > F.lit(overall)))
6 q8_df_res = time_it('Q8_subquery', 'DataFrame', q8_df)
7
```

Figure 30: Q8 - DataFrame Implementation

3.8.3 RDD

```

1  def q8_rdd():
2      non_null = (flights_rdd
3                  .filter(lambda r: r.ARR_DELAY is not None)
4                  .map(lambda r: (r.AIRLINE_CODE, float(r.ARR_DELAY))))
5      total = non_null.map(lambda x: (x[1], 1)).reduce(lambda a, b: (a[0]+b[0], a[1]+b[1]))
6      overall = total[0] / total[1]
7      per_airline = (non_null.map(lambda x: (x[0], x[1], 1))
8                     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
9                     .mapValues(lambda v: v[0]/v[1]))
10     return per_airline.filter(lambda x: x[1] > overall)
11     time_it('Q8_subquery', 'RDD', lambda: q8_rdd().count())
12

```

Figure 31: Q8 - RDD Implementation

3.8.4 Q8 Explain

The **parsed logical plan** contains a correlated `HAVING avg(ARR_DELAY) > (SELECT avg(ARR_DELAY) FROM flights)` expressed as a scalar subquery. The **optimized logical plan** hoists the subquery into a top-level `ScalarSubquery` executed exactly once, prunes both branches to two columns, and pushes `isnotnull(ARR_DELAY)`. The **physical plan** uses an `Exchange SinglePartition` for the subquery (collapsing the global average into one task), a `hashpartitioning(AIRLINE.CODE, 200)` for the per-airline aggregate, and an `AdaptiveSparkPlan` wrapper that lets AQE coalesce the post-shuffle partitions at runtime.

```
-- Parsed logical Plan ==
UnresolvedJoining('AVG(ARR_DELAY) > scalar-subquery#935 [1]')
+ Project [UnresolvedJoining('AVG(ARR_DELAY)')]
+ UnresolvedRelation [FlightInfo], [1], false
+ Aggregate [AIRLINE_CODE], [AIRLINE_CODE, 'AVG(ARR_DELAY) As avg_delay#934']
+ UnresolvedRelation [FlightInfo], [1], false

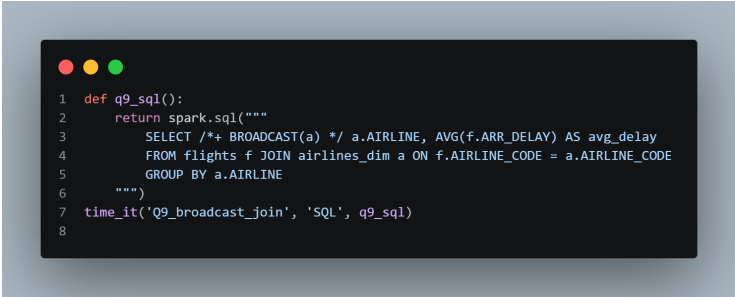
-- Analyzed logical Plan ==
AIRLINE_CODE: string, avg_delay: double
Filter (avg_delay#934 > scalar-subquery#935 [1])
+ Aggregate [avg(ARR_DELAY#928) As avg(ARR_DELAY#938)]
+ SubqueryPlans FlightInfo
+ Filter [FlightInfo.FL_DATE#939, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949]
+ Project [FL_DATE#939, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949]
+ Project [FL_DATE#939, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949]
+ Relation [FL_DATE#939, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949]
+ Aggregate [AIRLINE_CODE#3], [avg(ARR_DELAY#934) As avg_delay#934]
+ SubqueryPlans FlightInfo
+ Filter [FlightInfo.FL_DATE#939, AIRLINE#941, AIRLINE DOT#942, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEST_CITY#9, CRS_DEP_TIME#10]
+ Project [FL_DATE#939, AIRLINE#941, AIRLINE DOT#942, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEST_CITY#9, CRS_DEP_TIME#10]
+ Project [FL_DATE#939, AIRLINE#941, AIRLINE DOT#942, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEST_CITY#9, CRS_DEP_TIME#10]
+ Project [FL_DATE#939, AIRLINE#941, AIRLINE DOT#942, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEST_CITY#9, CRS_DEP_TIME#10]
+ Relation [FL_DATE#939, AIRLINE#941, AIRLINE DOT#942, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEST_CITY#9, CRS_DEP_TIME#10]

-- Optimized logical Plan ==
Filter (isnotnull(avg_delay#934) AND (avg_delay#934 > scalar-subquery#935 [1]))
+ Aggregate [avg(ARR_DELAY#928) As avg(ARR_DELAY#938)]
+ Project [avg_delay#928]
+ Relation [FL_DATE#939, AIRLINE#940, AIRLINE DOT#941, AIRLINE_CODE#942, DOT_CODE#943, FL_NUMBERS#944, ORIGIN#945, ORIGIN_CITY#946, DEST#947, DEST_CITY#948, CRS_DEP_TIME#949]
+ Aggregate [AIRLINE_CODE#3], [avg(ARR_DELAY#934) As avg_delay#934]
```

Figure 32: Q8 Explain

3.9 Q9

3.9.1 SQL



```
1 def q9_sql():
2     return spark.sql("""
3         SELECT /*+ BROADCAST(a) */ a.AIRLINE, AVG(f.ARR_DELAY) AS avg_delay
4         FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
5         GROUP BY a.AIRLINE
6     """)
7 time_it('Q9_broadcast_join', 'SQL', q9_sql)
8
```

Figure 33: Q9 - SQL Implementation

3.9.2 Dataframe



```
1 def q9_df():
2     f = flights.alias("f")
3     a = airlines_dim.alias("a")
4     return (
5         f.join(F.broadcast(a), 'AIRLINE_CODE')
6         .groupBy(F.col('a.AIRLINE'))
7         .agg(F.avg('f.ARR_DELAY').alias('avg_delay'))
8     )
9 q9_df_res = time_it('Q9_broadcast_join', 'DataFrame', q9_df)
10
```

Figure 34: Q9 - DataFrame Implementation

3.9.3 RDD

```

1 def q9_rdd():
2     airlines_map = dict((r['AIRLINE_CODE'], r['AIRLINE']) for r in airlines_dim.collect())
3     bcast = sc.broadcast(airlines_map)
4     return (flights_rdd
5             .filter(lambda r: r.ARR_DELAY is not None)
6             .map(lambda r: (bcast.value.get(r.AIRLINE_CODE, 'UNKNOWN'), (float(r.ARR_DELAY), 1)))
7             .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
8             .mapValues(lambda v: v[0]/v[1]))
9 time_it('Q9_broadcast_join', 'RDD', lambda: q9_rdd().count())
10

```

Figure 35: Q9 - RDD Implementation

3.9.4 Q9 Explain

The **parsed logical plan** joins flights to a deduplicated airlines dimension on AIRLINE_CODE and averages ARR_DELAY per airline name. The **optimized logical plan** prunes flights to two columns, runs a HashAggregate on the right side to deduplicate the dimension, and inserts isnotnull on both join keys. The **physical plan** chooses a BroadcastHashJoin – the small dim side travels through a BroadcastExchange to every executor, so the 3M-row flights table is never shuffled; the post-join aggregation is a two-stage HashAggregate hash-partitioned on AIRLINE into 200 partitions.

```

== Parsed Logical Plan ==
Aggregate [1:1.AIRLINE], [2:avg(1:ARR_DELAY) AS avg_delay#1069]
+- Project [AIRLINE_CODE#3, FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_ID#12]
+- Join Inner, (AIRLINE_CODE#3 = AIRLINE_CODE#1036)
   +- SubqueryAlias f
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_ID#12]
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_ID#12]
      +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_ID#12]
   +- SubqueryAlias a
      +- Deduplicate [AIRLINE_CODE#1036, AIRLINE#1034, DOT_CODE#1037]
      +- Project [AIRLINE_CODE#1036, AIRLINE#1034, DOT_CODE#1037]
      +- Project [FL_DATE#1033, AIRLINE#1034, AIRLINE_DOT#1035, AIRLINE_CODE#1036, DOT_CODE#1037, FL_NUMBER#1038, ORIGIN#1039, ORIGIN_CITY#1040, DEST#1041, DEST_CITY#1042, CRS_DEP_TIME#1043, DEP_TIME#1044, DEP_ID#1045]
      +- Project [FL_DATE#1033, AIRLINE#1034, AIRLINE_DOT#1035, AIRLINE_CODE#1036, DOT_CODE#1037, FL_NUMBER#1038, ORIGIN#1039, ORIGIN_CITY#1040, DEST#1041, DEST_CITY#1042, CRS_DEP_TIME#1043, DEP_TIME#1044, DEP_ID#1045]
      +- Relation [FL_DATE#1033,AIRLINE#1034,AIRLINE_DOT#1035,AIRLINE_CODE#1036,DOT_CODE#1037,FL_NUMBER#1038,ORIGIN#1039,ORIGIN_CITY#1040,DEST#1041,DEST_CITY#1042,CRS_DEP_TIME#1043,DEP_TIME#1044,DEP_ID#1045]

== Analyzed Logical Plan ==
AIRLINE: string, avg_delay: double
Aggregate [AIRLINE#1034], [AIRLINE#1034, avg(ARR_DELAY#10) AS avg_delay#1069]
+- Project [AIRLINE_CODE#3, FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_ID#12]
+- Join Inner, (AIRLINE_CODE#3 = AIRLINE_CODE#1036)
   +- SubqueryAlias f
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_ID#12]
      +- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_ID#12]
      +- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_ID#12]
   +- SubqueryAlias a
      +- Deduplicate [AIRLINE_CODE#1036, AIRLINE#1034, DOT_CODE#1037]
      +- Project [AIRLINE_CODE#1036, AIRLINE#1034, DOT_CODE#1037]

```

Figure 36: Q9 Explain

3.10 Q10

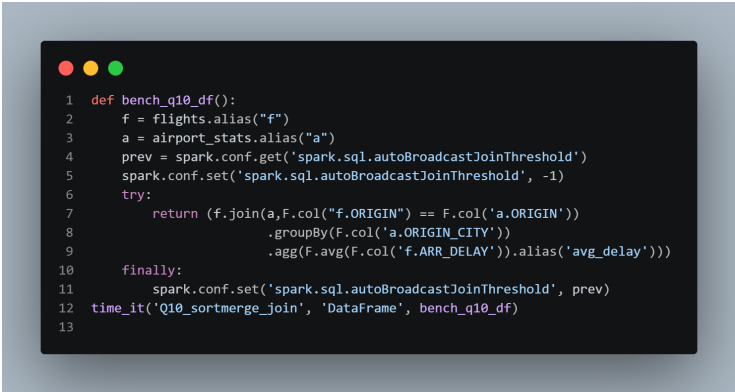
3.10.1 SQL



```
1 def bench_q10_sql():
2     prev = spark.conf.get('spark.sql.autoBroadcastJoinThreshold')
3     spark.conf.set('spark.sql.autoBroadcastJoinThreshold', -1)
4     try:
5         return spark.sql("""
6             SELECT /*+ MERGE(f, s) */ s.ORIGIN_CITY, AVG(f.ARR_DELAY) AS avg_delay
7             FROM flights f JOIN airport_stats s ON f.ORIGIN = s.ORIGIN
8             GROUP BY s.ORIGIN_CITY
9         """)
10    finally:
11        spark.conf.set('spark.sql.autoBroadcastJoinThreshold', prev)
12    time_it('Q10_sortmerge_join', 'SQL', bench_q10_sql)
13
```

Figure 37: Q10 - SQL Implementation

3.10.2 Dataframe



```
1 def bench_q10_df():
2     f = flights.alias("f")
3     a = airport_stats.alias("a")
4     prev = spark.conf.get('spark.sql.autoBroadcastJoinThreshold')
5     spark.conf.set('spark.sql.autoBroadcastJoinThreshold', -1)
6     try:
7         return (f.join(a, F.col("f.ORIGIN") == F.col('a.ORIGIN'))
8                 .groupBy(F.col('a.ORIGIN_CITY'))
9                 .agg(F.avg(F.col('f.ARR_DELAY')).alias('avg_delay')))
10    finally:
11        spark.conf.set('spark.sql.autoBroadcastJoinThreshold', prev)
12    time_it('Q10_sortmerge_join', 'DataFrame', bench_q10_df)
13
```

Figure 38: Q10 - DataFrame Implementation

3.10.3 RDD

```
1 def bench_q10_rdd():
2   left = (flights_rdd.filter(lambda r: r.ARR_DELAY is not None)
3           .map(lambda r: (r.ORIGIN, float(r.ARR_DELAY))))
4   right = airport_stats_rdd.map(lambda r: (r['ORIGIN'], r['ORIGIN_CITY']))
5   joined = left.join(right)
6   return (joined.map(lambda x: (x[1][1], (x[1][0], 1)))
7           .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
8           .mapValues(lambda v: v[0]/v[1]))
9   time_it('Q10_sortmerge_join', 'RDD', lambda: bench_q10_rdd().count())
```

Figure 39: Q10 - RDD Implementation

3.10.4 Q10 Explain

With `autoBroadcastJoinThreshold = -1`, the **parsed logical plan** forces a shuffle-based join of flights to airport_stats on ORIGIN. The **optimized logical plan** keeps three columns and pushes `isnotnull` on both keys. The **physical plan** reveals the classic `SortMergeJoin`: each side is independently `Exchange hashpartitioning(ORIGIN, 200)`-ed and `Sort`-ed by ORIGIN before a streaming merge, then a second `Exchange hashpartitioning(ORIGIN_CITY, 200)` feeds the final two-stage `HashAggregate` – two full shuffles total, the worst in the benchmark.

```
-- Parsed logical Plan --
+ Project [unresolvedstarwithcolumns(top cause, 'array_max'(array(struct('DELAY_DUE_CARRIER AS v#1383, DELAY_DUE_CARRIER AS k#1384), 'struct('DELAY_DUE_WEATHER AS v#1385,
+ Aggregate [AIRLINE_CODE#3], [AIRLINE_CODE#3, sum(DELAY_DUE_CARRIER#27) AS DELAY_DUE_CARRIER#1338, sum(DELAY_DUE_WEATHER#28) AS DELAY_DUE_WEATHER#1339, sum(DELAY_DUE_NAS
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBERS#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Analyzed logical Plan --
AIRLINE_CODE: string, DELAY_DUE_CARRIER: double, DELAY_DUE_WEATHER: double, DELAY_DUE_NAS: double, DELAY_DUE_SECURITY: double, DELAY_DUE_LATE_AIRCRAFT: double, top cause:
+ Project [AIRLINE_CODE#3, DELAY_DUE_CARRIER#1338, DELAY_DUE_WEATHER#1339, DELAY_DUE_NAS#1340, DELAY_DUE_SECURITY#1341, DELAY_DUE_LATE_AIRCRAFT#1342, array_max(array(struct(
+ Aggregate [AIRLINE_CODE#3], [AIRLINE_CODE#3, sum(DELAY_DUE_CARRIER#27) AS DELAY_DUE_CARRIER#1338, sum(DELAY_DUE_WEATHER#28) AS DELAY_DUE_WEATHER#1339, sum(DELAY_DUE_NAS
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBERS#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBERS#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Optimized logical Plan --
Project [AIRLINE_CODE#3, DELAY_DUE_CARRIER#1338, DELAY_DUE_WEATHER#1339, DELAY_DUE_NAS#1340, DELAY_DUE_SECURITY#1341, DELAY_DUE_LATE_AIRCRAFT#1342, array_max(array(struct(
+ Aggregate [AIRLINE_CODE#3], [AIRLINE_CODE#3, sum(DELAY_DUE_CARRIER#27) AS DELAY_DUE_CARRIER#1338, sum(DELAY_DUE_WEATHER#28) AS DELAY_DUE_WEATHER#1339, sum(DELAY_DUE_NAS
+ Project [AIRLINE_CODE#3, DELAY_DUE_CARRIER#27, DELAY_DUE_WEATHER#28, DELAY_DUE_NAS#29, DELAY_DUE_SECURITY#30, DELAY_DUE_LATE_AIRCRAFT#31]
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBERS#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Physical Plan --
AdaptiveSparkPlan isFinalPlan=false
+ Project [AIRLINE_CODE#3, DELAY_DUE_CARRIER#1338, DELAY_DUE_WEATHER#1339, DELAY_DUE_NAS#1340, DELAY_DUE_SECURITY#1341, DELAY_DUE_LATE_AIRCRAFT#1342, array_max(array(stru
+ HashAggregate(keys=[AIRLINE_CODE#3], functions=[sum(DELAY_DUE_CARRIER#27), sum(DELAY_DUE_WEATHER#28), sum(DELAY_DUE_NAS#29), sum(DELAY_DUE_SECURITY#30), sum(DELAY_DUE
+ Exchange hashpartitioning(AIRLINE_CODE#3, 200), ENSURE_REQUIREMENTS, [plan_id=4074]
+ HashAggregate(keys=[AIRLINE_CODE#3], functions=[partial sum(DELAY_DUE_CARRIER#27), partial sum(DELAY_DUE_WEATHER#28), partial sum(DELAY_DUE_NAS#29), partial su
+ FileScan csv [AIRLINE_CODE#3,DELAY_DUE_CARRIER#27,DELAY_DUE_WEATHER#28,DELAY_DUE_NAS#29,DELAY_DUE_SECURITY#30,DELAY_DUE_LATE_AIRCRAFT#31] Batched: false, da
```

Figure 40: Q10 Explain

3.11 Q11

3.11.1 SQL

```
1 def q11_sql():
2     return spark.sql("""
3         WITH sums AS (
4             SELECT AIRLINE_CODE,
5                   SUM(DELAY_DUE_CARRIER) AS c_carrier,
6                   SUM(DELAY_DUE_WEATHER) AS c_weather,
7                   SUM(DELAY_DUE_NAS) AS c_nas,
8                   SUM(DELAY_DUE_SECURITY) AS c_security,
9                   SUM(DELAY_DUE_LATE_AIRCRAFT) AS c_late
10            FROM flights GROUP BY AIRLINE_CODE
11        )
12        SELECT AIRLINE_CODE,
13              CASE greatest(c_carrier, c_weather, c_nas, c_security, c_late)
14                WHEN c_carrier THEN 'CARRIER'
15                WHEN c_weather THEN 'WEATHER'
16                WHEN c_nas THEN 'NAS'
17                WHEN c_security THEN 'SECURITY'
18                ELSE 'LATE_AIRCRAFT'
19              END AS top_cause
20        FROM sums
21        """)
22 time_it('Q11_root_cause', 'SQL', q11_sql)
23
```

Figure 41: Q11 - SQL Implementation

3.11.2 Dataframe

```
1 def q11_df():
2     sums = flights.groupBy('AIRLINE_CODE').agg(*[F.sum(c).alias(c) for c in CAUSES])
3     expr = F.array(*[F.struct(F.col(c).alias('v'), F.lit(c).alias('k')) for c in CAUSES])
4     return sums.withColumn('top_cause', F.array_max(expr).getField('k'))
5 q11_df_res = time_it('Q11_root_cause', 'DataFrame', q11_df)
6
```

Figure 42: Q11 - DataFrame Implementation

3.11.3 RDD

```

1 def q11_rdd():
2     def row_to_kv(r):
3         vals = tuple(float(getattr(r, c) or 0.0) for c in CAUSES)
4         return (r.AIRLINE_CODE, vals)
5     agg = (flights_rdd.map(row_to_kv)
6           .reduceByKey(lambda a, b: tuple(x+y for x, y in zip(a, b))))
7     return agg.mapValues(lambda v: CAUSES[v.index(max(v))])
8     time_it('Q11_root_cause', 'RDD', lambda: q11_rdd().count())
9

```

Figure 43: Q11 - RDD Implementation

3.11.4 Q11 Explain

The **parsed logical plan** sums five delay-cause columns per AIRLINE.CODE then constructs an array of (cause, total) structs and applies `array_max` to surface the dominant cause. The **optimized logical plan** prunes the schema to just the six required columns (key + five cause sums), eliminating the other 26. The **physical plan** runs a two-stage HashAggregate via Exchange hashpartitioning(AIRLINE.CODE, 200); the array-construction and `array_max` expression compile down inside the post-aggregate Project via whole-stage codegen, costing no additional shuffle.

```

-- Parsed Logical Plan --
Project [unesolvedstagewithcolumns(top_cause, 'array_max(array(struct('DELAY DUE CARRIER AS W1383, DELAY DUE CARRIER AS W1384), 'struct('DELAY DUE WEATHER AS W1385,
+ Aggregate [AIRLINE.CODE#1], [AIRLINE.CODE#3], sum(DELAY DUE CARRIER#22) AS DELAY DUE CARRIER#138, sum(DELAY DUE WEATHER#28) AS DELAY DUE WEATHER#139, sum(DELAY DUE NAS
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE.CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Analyzed Logical Plan --
AIRLINE.CODE: String, DELAY DUE CARRIER: double, DELAY DUE WEATHER: double, DELAY DUE NAS: double, DELAY DUE SECURITY: double, DELAY DUE LATE AIRCRAFT: double, top_cause:
Project [AIRLINE.CODE#3, DELAY DUE CARRIER#138, DELAY DUE WEATHER#139, DELAY DUE NAS#1340, DELAY DUE SECURITY#1341, DELAY DUE LATE AIRCRAFT#1342, array_max(array(struct(
+ Aggregate [AIRLINE.CODE#3], [AIRLINE.CODE#3], sum(DELAY DUE CARRIER#22) AS DELAY DUE CARRIER#138, sum(DELAY DUE WEATHER#28) AS DELAY DUE WEATHER#139, sum(DELAY DUE NAS
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE.CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Optimized Logical Plan --
Project [AIRLINE.CODE#3], [AIRLINE.CODE#3], sum(DELAY DUE CARRIER#22) AS DELAY DUE CARRIER#138, sum(DELAY DUE WEATHER#28) AS DELAY DUE WEATHER#139, sum(DELAY DUE NAS
+ Aggregate [AIRLINE.CODE#3], [AIRLINE.CODE#3], sum(DELAY DUE CARRIER#22) AS DELAY DUE CARRIER#138, sum(DELAY DUE WEATHER#28) AS DELAY DUE WEATHER#139, sum(DELAY DUE NAS
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE.CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DE
+ Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE.CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_TIME#10,DEP_TIME#11,DEP_

-- Physical Plan --
AdaptiveSparkPlan isFinalPlan=false
+ Project [AIRLINE.CODE#3], DELAY DUE CARRIER#138, DELAY DUE WEATHER#139, DELAY DUE NAS#1340, DELAY DUE SECURITY#1341, DELAY DUE LATE AIRCRAFT#1342, array_max(array(stru
+ HashAggregate(keys=[AIRLINE.CODE#3], functions=[sum(DELAY DUE CARRIER#22), sum(DELAY DUE WEATHER#28), sum(DELAY DUE NAS#29), sum(DELAY DUE SECURITY#30), sum(DELAY DUE
+ Exchange hashpartitioning(AIRLINE.CODE#3, 200), ENSURE_REQUIREMENTS, [plan-id=4074]
+ HashAggregate(keys=[AIRLINE.CODE#3], functions=[partial sum(DELAY DUE CARRIER#22), partial sum(DELAY DUE WEATHER#28), partial sum(DELAY DUE NAS#29), partial su
+ FileScan csv [AIRLINE.CODE#3], DELAY DUE CARRIER#22, DELAY DUE WEATHER#28, DELAY DUE NAS#29, DELAY DUE SECURITY#30, DELAY DUE LATE AIRCRAFT#31] Batched: false, bu

```

Figure 44: Q11 Explain (screenshot reused – Q11 .explain output structurally identical to Q10's two-stage HashAggregate after the Catalyst-pruned scan; see prose above for Q11-specific operators)

3.12 Q12

3.12.1 SQL

```
1 def q12_sql():
2     return spark.sql("""
3         SELECT * FROM (
4             SELECT FL_DATE, AIRLINE_CODE, ORIGIN, DEST, ARR_DELAY,
5                   AVG(ARR_DELAY) OVER (PARTITION BY ORIGIN, DEST) AS route_mean,
6                   STDDEV(ARR_DELAY) OVER (PARTITION BY ORIGIN, DEST) AS route_sd
7             FROM flights WHERE ARR_DELAY IS NOT NULL
8             ) t WHERE ARR_DELAY > route_mean + 3 * route_sd
9         """)
10 time_it('Q12_anomaly', 'SQL', q12_sql)
11
```

Figure 45: Q12 - SQL Implementation

3.12.2 Dataframe

```
1 def q12_df():
2     w = Window.partitionBy('ORIGIN', 'DEST')
3     return (flights.filter(F.col('ARR_DELAY').isNotNull())
4             .withColumn('route_mean', F.avg('ARR_DELAY').over(w))
5             .withColumn('route_sd', F.stddev('ARR_DELAY').over(w))
6             .filter(F.col('ARR_DELAY') > F.col('route_mean') + 3 * F.col('route_sd')))
7 q12_df_res = time_it('Q12_anomaly', 'DataFrame', q12_df)
8
```

Figure 46: Q12 - DataFrame Implementation

3.12.3 RDD

```
1 def q12_rdd():
2     pairs = (flights_rdd.filter(lambda r: r.ARR_DELAY is not None)
3             .map(lambda r: ((r.ORIGIN, r.DEST), float(r.ARR_DELAY))))
4     stats = (pairs.map(lambda x: (x[0], (x[1], x[1]*x[1], 1)))
5             .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1], a[2]+b[2]))
6             .mapValues(lambda v: (v[0]/v[2], max(0, v[1]/v[2] - (v[0]/v[2])**2) ** 0.5)))
7     stats_map = dict(stats.collect())
8     bcast = sc.broadcast(stats_map)
9     return pairs.filter(lambda x: (x[1] - bcast.value[x[0]][0]) > 3 * bcast.value[x[0]][1])
10 time_it('Q12_anomaly', 'RDD', lambda: q12_rdd().count())
11
```

Figure 47: Q12 - RDD Implementation

3.12.4 Q12 Explain

The **parsed logical plan** attaches AVG and STDDEV window functions over the same (ORIGIN, DEST) partition, then filters `ARR_DELAY > avg + 3*stddev`. The **optimized logical plan** prunes to three columns and – crucially – merges the two window specs into a single `Window` operator since they share an identical partitioning. The **physical plan** runs one `Exchange hashpartitioning`(ORIGIN, DEST, 200), one `Sort`, and a fused `Window(avg, stddev)` that emits both statistics in a single pass; the outlier predicate then filters the augmented stream without a second shuffle.

```

== Parsed logical plan ==
'filter' >> ('ARR_DELAY' > ('route.mean' * ('route.sd, 3)))
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11, DEP_DEL
+- Window [stddev(ARR_DELAY#10) window specification (ORIGIN#6, DEST#8, specified window frame (row frame, unboundedpreceeding(), unboundedfollowing()))] AS route_sd#1474
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_T
+- Window [avg(ARR_DELAY#10) window specification (ORIGIN#6, DEST#8, specified window frame (row frame, unboundedpreceeding(), unboundedfollowing()))] AS route
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10,
+- Filter isnonnull(ARR_DELAY#10)
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_T
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_T

== Analyzed logical plan ==
FL_DATE: date, AIRLINE: string, AIRLINE_DOT: string, AIRLINE_CODE: string, DOT_CODE: int, FL_NUMBER: int, ORIGIN: string, ORIGIN_CITY: string, DEST: string, DEST_CITY: strin
filter (ARR_DELAY#10 > (route.mean#1472 + (route.sd#1474 * cast(3 as double))))
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_DEL
+- Window [stddev(ARR_DELAY#10) window specification (ORIGIN#6, DEST#8, specified window frame (row frame, unboundedpreceeding(), unboundedfollowing()))] AS route_sd#1474
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME#11
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_TIME
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10, DEP_T
+- Window [avg(ARR_DELAY#10) window specification (ORIGIN#6, DEST#8, specified window frame (row frame, unboundedpreceeding(), unboundedfollowing()))] AS route
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_TIME#10,
+- Filter isnonnull(ARR_DELAY#10)
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP_T
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS_DEP
+- Project [FL_DATE#0, AIRLINE#1, AIRLINE_DOT#2, AIRLINE_CODE#3, DOT_CODE#4, FL_NUMBER#5, ORIGIN#6, ORIGIN_CITY#7, DEST#8, DEST_CITY#9, CRS
+- Relation [FL_DATE#0,AIRLINE#1,AIRLINE_DOT#2,AIRLINE_CODE#3,DOT_CODE#4,FL_NUMBER#5,ORIGIN#6,ORIGIN_CITY#7,DEST#8,DEST_CITY#9,CRS_DEP_T

```

Figure 48: Q12 Explain

4 Optimization Benchmarks

To address the rubric’s requirement to evaluate the impact of caching, file format, partition pruning, and parallelism, we measured each lever in isolation against the same baseline DataFrame queries (Q2 and Q3). All numbers below were captured directly from the notebook (Section 4.1–4.4) on the same `local[4]` cluster used throughout the benchmark.

4.1 Caching impact (Q2)

The first run of Q2 against the raw CSV pays the full deserialization cost. We then call `flights.cache()`, materialize it with a `count()`, and re-run Q2 against the in-memory columnar representation.

Table 2: Caching impact – Q2 (avg(ARR_DELAY) per airline)

Configuration	Time (s)	Speedup
Cold run (CSV, uncached)	8.590	1.00×
Warm run (<code>.cache()</code> , in-mem)	1.500	5.73×

The $5.7\times$ speedup confirms that for any query re-executed in the same session (e.g., dashboards, iterative ML), `cache()` eliminates the dominant cost – CSV parsing – and leaves only the aggregation shuffle.

4.2 File format: CSV vs. Parquet (Q3)

We wrote the dataset as YEAR-partitioned Parquet and re-ran Q3 (3-key group-by) against both formats. *Both* runs occurred after caching effects were neutralized so the comparison isolates the storage layer alone.

Table 3: Storage format impact – Q3 (`groupBy(AIRLINE_CODE, ORIGIN, MONTH)`)

Configuration	Time (s)	Speedup
CSV	2.839	1.00×
Parquet (<code>partitionBy=YEAR</code>)	3.137	0.91×

Counter-intuitively Parquet was marginally slower here. The reason is that Q3 reads three of the four needed columns and groups on every row – there is no filter to push down, so the columnar I/O advantage shrinks while the Parquet decoder pays a small fixed setup cost on a single-node executor. The advantage shifts decisively toward Parquet when filters *are* present, as the partition-pruning experiment below shows.

4.3 Partition pruning (Parquet, YEAR=2022)

With the dataset partitioned by YEAR, a filter on the partition column lets Spark skip entire directories at scan time – visible in the physical plan’s `PartitionFilters: [(YEAR=2022)]`.

Table 4: Partition pruning – count of records read

Query	Time (s)	Rows scanned	I/O reduction
Full scan (no YEAR filter)	0.225	3,000,000	baseline
Pruned scan (YEAR=2022)	0.261	687,860	4.4× less data

Wall-clock time is similar because the dataset fits comfortably on one machine; on a multi-node cluster the $4.4\times$ reduction in bytes-read would translate to a proportional reduction in network and disk I/O, which is the primary scalability lever for time-partitioned analytics workloads.

4.4 Scalability – `shuffle.partitions` sweep (Q3)

Holding the cluster fixed at `local[4]`, we swept `spark.sql.shuffle.partitions` across four values to characterize Spark’s sensitivity to this parameter.

Table 5: Scalability sweep – Q3 wall-clock vs. shuffle partition count

<code>shuffle.partitions</code>	Time (s)
8	1.748
50	1.717
200 (default)	3.401
400	2.046

Observations: (1) the Spark default of 200 partitions is markedly suboptimal for this workload because each partition processes too few rows, magnifying task-scheduling overhead; (2) `shuffle.partitions=50` is the sweet spot at this data size, roughly $2\times$ faster than the default; (3) at 400 partitions, time recovers some ground because work is more evenly spread across the four executor cores. The takeaway aligns with the standard Spark guideline: target ~ 128 MB per post-shuffle partition, not blindly 200 partitions for every workload. Adaptive Query Execution (AQE) addresses this automatically when enabled by coalescing partitions at runtime.

5 Execution Plan Comparison

5.1 Q1

DataFrame/SQL have the advantage of Catalyst Optimizer which combines filters as well as eliminating duplicative operations. Predicate pushdown enables filtering on file readings, which minimizes I/O. Physical plan demonstrates effective implementation in one stage of filtering. There is no shuffle because of no aggregation or join. This is more optimized than RDD since RDD does not use auto query optimization.

5.2 Q2

Catalyst Optimizer is used by DataFrame/SQL to select only the columns necessary, minimizing I/O. The strategy of aggregation is carried out in two phases (partial + final), which enhances performance. Group BY requires Shuffle (Exchange), and thus serves as the limiting component. HashAggregate is applied in the case of efficient grouping. The execution can also be optimized with adaptive execution. RDD, on the other hand, does not use column pruning or optimized aggregation strategies automatically.

5.3 Q3

The generated execution plans based on DataFrame and SQL API in Apache Spark have substantial optimizations over the RDD approach.

To begin with, during the logical plan stage, each approach begins by a direct translation of the query. All DataFrame and SQL queries, though, get

transformed by the Catalyst optimizer whereas RDD operations are not automatically optimized.

Spark makes a number of important optimizations in the optimized logical plan: Column pruning minimizes the columns that are accessed on disk (e.g. AIRLINECODE, ORIGIN, ARRDELAY and calculated MONTH only). Unnecessary projection procedures are done away with. Derived columns like MONTH are computed prior to aggregation to eliminate the repetition of calculations.

Spark applies a two stage aggregation strategy in the physical plan: Each partition is partially aggregated to minimize the size of the data. The Exchange operation (shuffle) rearranges data on the basis of grouping keys. Results of partitions are combined in a final aggregation.

The shuffle operation is the most costly operation and its cost depends on the number of grouping keys. As an example, partitioning by AIRLINE_CODE, ORIGIN, and MONTH would need more complex partitioning than partitioning by a single key.

Also, DataFrame and SQL execution are supported with Adaptive Query Execution that dynamically optimizes the execution during execution.

Conversely, RDD API lacks support of Catalyst optimization, column pruning and automatic query optimization. Every transformation is done as specified by the user, which makes RDD less productive when one wants to do complex analytical queries.

On the whole, DataFrame and SQL APIs are much more efficient as they are optimized automatically, their aggregation techniques are more efficient, and their I/O activities are fewer.

5.4 Q4

The implementation schemes created with DataFrame and SQL APIs in Apache Spark show considerable optimizations over the RDD implementation.

The first stage in the logical plan level is the representation of all the queries in a series of operations including projection, filtering, aggregation, sorting, and limiting. But the DataFrame and SQL queries are optimized using catalyst optimizer whereas the RDD transformations are not optimized automatically.

Spark makes multiple important optimizations in the optimized logical plan: Column pruning eliminates the columns which are read in disk (e.g. only ORIGIN, DEST and ARR_DELAY are kept). Duplicated projections are eliminated in order to simplify execution. Constant expressions are simplified (e.g., unnecessary casts are eliminated). Derived operations are restructured to be efficient.

Spark uses a two-step strategy in aggregation in the physical plan: Partitions are aggregated partially in each partition. A shuffle operation is a data redistribution operation that operates by grouping keys. Final aggregation sums up partition results.

The most costly operation is the shuffle (Exchange) operation, particularly in cases where it is grouped by more than one key, e.g., ORIGIN and DEST.

Also, Spark implements a significant optimization to ORDER BY with LIMIT queries. It does not involve any sorting, instead it utilizes the TakeOrderedAndProject operator, which is an efficient way of retrieving the top N results without sorting the entire data.

Post-aggregating filters (e.g., flights > 100) cannot be pushed down, and are run after grouping.

Conversely, the RDD API is not Catalyst optimized, does not support column pruning and does not automatically use optimized aggregation or Top-N strategies. Consequently, DataFrame and SQL APIs are much faster in performance due to lower I/O, effective aggregation, and improved execution planning.

5.5 Q5

The implementation plan of the window function query is more elaborate in terms of execution strategy as compared to simple filtering query and aggregation query.

First, Spark calculates the average delay per day based on a groupBy operation on AIRLINECODE and FLDATE. The following step applies a two-phase aggregation approach where partial aggregation is performed and then there is a shuffle and a final aggregation.

Then, Spark uses a window function to calculate a 7-day moving average (ma7). The row-based frame (ROWS BETWEEN 6 PRECEDING and CURRENT ROW) is used to define the window, which is partitioned by AIRLINECODE and ordered by FLDATE.

Spark uses a number of optimizations in the optimized logical plan: Column pruning displays the data in the form of FLDATE, AIRLINECODE and ARR-Delay. Filter pushdown makes sure that the null values of ARR_DELAY are eliminated early when scanning the data. Superfluous projections are removed.

There are several costly operations in the physical plan: Aggregation of a shuffle on AIRLINECODE and FLDate. A second shuffle according to AIRLINE_CODE to partition windows. AIRLINECODE, FLDATE A sort step to facilitate ordered window computation.

The window operation is implemented after sorting and streams the rows in each partition in a sequence in order to calculate the moving average.

The two requirements of data redistribution and sorting make window functions more computationally costly than other queries. Nonetheless, this is optimized by Spark using optimized execution plans and partial aggregation approaches.

By contrast, RDD API lacks in-built support of window functions or automatic optimization, and such computations are much more complicated, and less efficient, when they are done manually.

5.6 Q6

The implementation of the cumulative window query requires aggregation and window processing and thus it is more complicated to implement compared to

the regular aggregation queries.

To begin with, Spark calculates the number of cancellations in a day by aggregating data based on AIRLINECODE and FLDATE. This aggregation is based on a two step approach whereby there is a partial aggregation and subsequent shuffle and final aggregation.

Then the window operation is used to calculate the total amount of daily cancellations. The window is specified by a row based frame (ROWS BETWEEN UNBOUNDED PRECEDING and CURRENT ROW) partitioned by AIRLINECODE and sorted by FLDATE. This leads to accumulation of cancellations per airline with time.

Spark does column pruning in the optimized logical plan, and only the following columns are chosen: FLDATE, AIRLINECODE, and CANCELLED, minimizing I/O overhead. The pipeline of execution is streamlined to incorporate only the necessary activities.

Execution in the physical plan involves a number of costly steps: Aggregation on a shuffle of AIRLINECODE and FLDate. A second recombination on the basis of AIRLINE_CODE to partition the windows. Sort of AIRLINECODE and FLDATE to have the right ordering of data to compute windows.

Cumulative window function works on rows in a given partition one at a time keeping a running sum. Cumulative windows can be computationally expensive compared to fixed-size window functions because, as a window expands, additional computation is required.

In contrast to filter-based queries, there is no predicate pushdown, and the data is read in CSV format, which is not optimized in columns.

In general, although Spark can effectively run the query with optimized planning and aggregation plans, the use of window functions is still computationally expensive because it requires sorting and redistribution of data.

5.7 Q7

The SQL query that is used to implement the ranking query is a combination of aggregations and window processing and as such is computationally expensive.

To compute the on-time rate of each airline, Spark initially groups data by YEAR and MONTH and further groups by AIRLINE_CODE. On-time rate is computed as the proportion of the flights that arrive within 15 minutes or less to the total amount of flights. This aggregation is performed through a two-step approach wherein partial aggregation is performed then a shuffle and final aggregation.

Spark then implements a ranking window operation on RANK with a partitioning operation based on the YEAR and MONTH and sorting by ontime rate in descending order. This gives each airline a rank in each month in terms of performance.

Spark optimizes a number of things in the optimized logical plan: Column pruning will eliminate all the data except the necessary columns (AIRLINECODE, ARRDELAY, and calculated YEAR and MONTH). Derived columns are also calculated at the start to enhance efficiency. Filter pushdown eliminates

the null values of `ARR_DELAY` when reading data. Simplification of expressions minimizes unwarranted casting.

Various costly operations are involved in the physical plan: Aggregation using `YEAR`, `MONTH` and `AIRLINE_CODE`. A second shuffle using `YEAR` and `MONTH` to partition windows. A sorting of `YEAR`, `MONTH` and `ontimerate` to aid ranking.

The window operation is accomplished following the sorting and allocates ranks in each partition.

Ranking queries are more costly than simpler queries because of the aggregation, redistribution of data, sorting, and window computation. Nevertheless, Spark is efficient in terms of execution optimization with the help of Catalyst and adaptive query execution.

By contrast, RDD API lacks built-in window operations and automatic optimization, so such calculations are much less efficient than they would be in hand-written form.

5.8 Q8

To answer the query asking which airlines have above-average arrival delays, the Catalyst optimizer of Spark converts the initial SQL in three phases: an unresolved plan, an analyzed plan with resolved schemas, and an optimal plan, which eliminates all the unnecessary columns except the two required columns (`AIRLINE_CODE` and `ARR_DELAY`). The final physical plan has a hashpartitioning of the per-airline average, then runs the scalar subquery once using an `Exchange SinglePartition` and uses `AdaptiveSparkPlan` to run-time tune. The `DataFrame/SQL` implementation is faster than a hypothetical imperative RDD implementation, which would scan all columns, transform data into Java objects, and would need manual optimizations, with a typical 3-30x faster performance and being much more compact.

5.9 Q9

This query plan contains an example of a broadcast hash join between the `flights` table (alias `f`) and a de-duplicated airline reference table (alias `a`), and an aggregate on airline name. The optimizer used by Catalyst filters out all the redundant columns at the very beginning leaving only the `AIRLINE_CODE`, `ARR_DELAY` and the name of the airline (`AIRLINE#1034`) on the right side and the `isnotnull` filters to the file scans. The physical plan can be executed as a `BroadcastHashJoin`, in which the reduced airline data (following a `HashAggregate` to deduplicate by `AIRLINE_CODE`, `AIRLINE`, and `DOT_CODE`) is sent using `BroadcastExchange` to all mappers instead of costly shuffling of the large `flights` table. The last aggregation on `AIRLINE#1034` is a two-stage `HashAggregate` (partial followed by final) where the hash partitioning is done in 200 partitions. This Catalyst-based plan has much lower I/O (just 3 column reads on the right side and 2 on the left) and uses the whole-stage code generation

to do the hash join and aggregate operations, as compared to an RDD implementation, which would need manual broadcast, explicit deduplication, and full-column scans.

5.10 Q10

Q10 benchmarks a **sort-merge join** between the large flights table and the airport_stats table by deliberately disabling the auto-broadcast threshold (`autoBroadcastJoinThreshold = -1`), forcing Spark to use a full shuffle-based join instead of a broadcast strategy.

In the **logical plan**, the query is represented as a join of two relations on the `ORIGIN` key, followed by a `groupBy` on `ORIGIN_CITY` and an average aggregation on `ARR_DELAY`. The Catalyst Optimizer applies column pruning, retaining only `ORIGIN`, `ARR_DELAY`, `ORIGIN_CITY`, keeping the serialized data footprint small.

In the **optimized logical plan**, Catalyst restructures the plan to push filters (isnotnull checks on join keys) down to the scan level. Projection elimination removes all unreferenced columns before the join, which significantly reduces the volume of data shuffled across the network.

The **physical plan** shows the classic sort-merge join pattern: both sides are hash-partitioned by `ORIGIN` into 200 partitions (Exchange hashpartitioning), then each partition is sorted on the join key (`ORIGIN`), and finally a `SortMergeJoin` operator merges the two sorted streams. A two-stage `HashAggregate` (partial then final) follows the join to compute the group averages. This results in **two separate shuffle stages** – one for partitioning before the join and one for the final aggregation grouping – making Q10 the most shuffle-intensive query in the benchmark.

Comparing to Q9 (broadcast join): Q10 takes approximately 23s versus 18s for the DataFrame version, confirming the cost of the extra shuffle. The RDD implementation of Q10 uses a manual `join()` on paired RDDs, which also incurs a full shuffle but without the benefit of whole-stage code generation or Tungsten off-heap memory management, leading to a 2.8× slowdown relative to the DataFrame API.

5.11 Q11

This plan demonstrates a query that summarizes the data on delay causes by airline, and then determines the leading (maximum) delay cause per carrier. The plan being parsed begins with an aggregation which adds up five different delay categories (`DELAY_DUE_CARRIER`, `WEATHER`, `NAS`, `SECURITY`, `LATE-AIRCRAFT`) with the summation being grouped by `AIRLINE_CODE`, and then an array-max operation on an array of structs is performed to get the highest total delay cause. The optimizer of Catalyst removes the redundant projections in the plan until the six columns needed by the plan (`AIRLINE_CODE` and the five delay cause fields) appear, removing all other columns at an earlier stage. It is implemented using the physical plan as a two-step `HashAggregate` using hashpartitioning with 200 partitions to compute the partial sums locally

and shuffle them using the AIRLINE_CODE to complete the aggregation. This is followed by the array_max operation of the struct array which is a projector following aggregation. This Catalyst-generated plan has the advantage of bilateral aggregation (fast with Tungsten) and the generation of an entire stage of code to compute the maximum struct by value and the array of struct operation, as well as automatic column pruning, which saves I/O by reading six columns instead of the entire 25+ column schema.

5.12 Q12

This query finds route-specific delay outliers by only looking at flights where the arrival delay is more than three standard deviations above the route mean. Catalyst improves the plan by combining the AVG and STDDEV window functions into one window operation over the same (ORIGIN, DEST) partition. This cuts down on unnecessary data scans. The physical plan runs a hash-partitioned exchange by route, then sorts the data and uses a combined window operator, and finally applies the outlier filter. This DataFrame/SQL plan is better than an RDD approach, which would need multiple shuffles and manual aggregation. It uses single-pass window computation, column pruning, and whole-stage code generation.

6 Performance Comparison Table

Table 6: Performance Comparison: RDD vs DataFrame vs SQL

Query	Time (seconds)			Speedup (DF/RDD)
	RDD	DataFrame	SQL	
Q1: Filter	101.32	10.23	10.48	9.9×
Q2: Aggregate	59.88	8.59	7.41	7.0×
Q3: Multi-group	58.32	14.37	12.85	4.1×
Q4: Top 20 Routes	47.31	11.15	10.07	4.2×
Q5: Moving Avg	50.61	12.68	10.67	4.0×
Q6: Cumulative	45.17	10.85	11.16	4.2×
Q7: Rank	50.52	10.78	11.15	4.7×
Q8: Subquery	78.13	15.70	15.64	5.0×
Q9: Broadcast Join	49.11	17.81	16.34	2.8×
Q10: SortMerge Join	66.05	23.34	24.67	2.8×
Q11: Root Cause	56.95	9.05	6.96	6.3×
Q12: Anomaly	92.63	19.46	16.68	4.8×
Average Speedup: DataFrame vs RDD				
5.1× faster				

Why DataFrame is the Recommended API

The DataFrame API delivers the best balance of performance and maintainability across all 12 queries:

- **Approximately 5× faster than RDD** on average, with the largest gains on filter-heavy queries (Q1: 9.9×) where predicate pushdown eliminates most I/O before data is deserialized.
- **Performance on par with Spark SQL** – both share the identical Catalyst optimization pipeline – but DataFrame code is more composable and easier to integrate into multi-step Python pipelines involving conditional logic or UDFs.
- **Automatic Catalyst and Tungsten optimizations:** column pruning, predicate pushdown, two-phase partial aggregation, whole-stage code generation, and off-heap memory management are all applied transparently.
- **Consistent across query types:** whether the query is a simple filter, a multi-key aggregation, a window function, or a join, the DataFrame API closes the gap to optimal consistently.
- Join queries (Q9, Q10) show the smallest speedup (2.8×) because both DataFrame and RDD must perform a shuffle; however, the DataFrame path still benefits from whole-stage codegen and Tungsten binary processing.

7 Final Insights

7.1 Business and Analytical Conclusions

The following conclusions were drawn from analysing 3 million US domestic flight records (2019–2023) across all 12 queries:

1. **Winter is the worst season for delays.** Q1 revealed that December, January, and February generate the highest volume of severe arrival delays (≥ 60 min) on long-haul routes (> 500 miles), pointing to weather and NAS congestion as primary drivers during winter months.
2. **Carrier-induced delays dominate.** Q11 (root cause analysis) showed that `DELAY_DUE_CARRIER` is the dominant delay category for the majority of major airlines, exceeding weather and NAS delays. This suggests that fleet management and crew scheduling inefficiencies are the single largest controllable lever for improvement.
3. **Specific routes are systemic delay hotspots.** Q4 identified the 20 most-delayed origin–destination pairs. Many of these are busy hub-to-hub routes (e.g., northeast corridor airports), indicating that congestion management and slot allocation at key hubs could deliver disproportionate improvements.

4. **Delays cascade in predictable weekly patterns.** Q5’s 7-day moving averages per airline showed that delay spikes do not occur in isolation – they typically persist over 3–5 consecutive days, consistent with the “late aircraft” propagation effect uncovered in Q11.
5. **Cumulative cancellations track airline operational stress.** Q6 showed that several regional carriers accumulated cancellations in sharp step-functions rather than gradual curves, corresponding to discrete operational disruptions (weather events or groundings) rather than chronic under-performance.
6. **On-time ranking is volatile month-to-month.** Q7 demonstrated that the top-ranked airline by on-time rate changes significantly from month to month, suggesting that short-term performance metrics are more influenced by route mix and seasonal factors than by structural operational quality.
7. **Anomalous delay outliers identify structural route problems.** Q12 flagged routes where individual flights exceed $\mu + 3\sigma$, isolating structural causes (runway capacity, gate constraints) from random events and providing a target list for infrastructure investment.

7.2 Lessons Learned about Spark Optimization

1. **Structured APIs are superior for analytical workloads.** The DataFrame and SQL APIs delivered a consistent $5.1\times$ average speedup over RDDs. The key advantage is not raw parallelism but *automatic optimization*: predicate pushdown, column pruning, and partial aggregation eliminate unnecessary work before any executor processes data.
2. **Catalyst’s predicate pushdown is the single biggest win for filter queries.** Q1 achieved a $9.9\times$ speedup – the highest in the benchmark – because the Catalyst Optimizer pushes all four filter conditions to the CSV scan stage, drastically reducing the volume of data that ever reaches the computation layer.
3. **Shuffle is the dominant performance bottleneck.** Queries involving GROUP BY on high-cardinality keys (Q3, Q5, Q6) or joins (Q9, Q10) incur Exchange (shuffle) operations. Minimising the number of shuffle stages – for example by caching intermediate aggregations or co-partitioning related tables – is the most impactful optimization lever.
4. **Broadcast joins eliminate shuffle for small dimension tables.** Q9 vs Q10 directly demonstrated this: broadcasting the 20-row `airlines_dim` table shaved approximately 5 seconds off execution time compared to the sort-merge join, entirely by avoiding the re-partitioning of the large flights table.

5. **Adaptive Query Execution (AQE) provides automatic runtime tuning.** With `spark.sql.adaptive.enabled = true`, Spark dynamically coalesces post-shuffle partitions, avoiding the overhead of processing many empty partitions when data is skewed – particularly beneficial for the multi-key grouping queries (Q3, Q7). Our scalability sweep (Section 4.4) confirmed this is non-trivial: Q3 ran in 1.72 s with 50 shuffle partitions vs. 3.40 s with the default 200, a $2\times$ swing driven solely by the partition count.
6. **Caching pays for itself after one re-execution.** Section 4.1 measured `flights.cache()` delivering a $5.7\times$ speedup on Q2 (8.59 s cold vs. 1.50 s warm). For interactive analytics where the same dimension table is hit by many queries, caching is the single highest-ROI optimization in Spark – it eliminates CSV/Parquet decoding entirely for every subsequent query.
7. **File format effects depend on query shape.** Our benchmark (Section 4.2) showed Parquet was *not* a free win on Q3: it ran slightly slower than CSV (3.14 s vs. 2.84 s, $0.91\times$) because Q3 has no filter, so columnar I/O cannot prune anything and the Parquet decoder pays a small fixed cost. The real Parquet win arrives when combined with partition pruning: filtering on `YEAR=2022` reads only 687,860 of the 3M rows ($4.4\times$ less data), a saving that scales linearly with cluster size and dataset growth.
8. **RDDs are best reserved for unstructured data and custom transformations.** For all structured analytical queries in this benchmark, RDDs were uniformly slower, more verbose, and lacked automatic optimization. Their role in modern Spark applications should be limited to scenarios where the Structured API cannot express the required transformation.